

Άσκηση 1

README

Ομάδα

Ιωάννης Γρυπιώτης 1115202100028

Ιωάννης Κούβελας 1115202100070

Πάντιος Ματιάτος 1115202100098

Περιεχόμενα

1	Υλοποίηση	1
1.1	Ανάθεση Συναρτήσεων	1
1.2	Γενικά	1
1.3	Πώς να τρέξετε το εκτελέσιμο	2
1.4	Επεξηγήσεις	2
1.5	Σχεδιαστικές Επιλογές	2
1.6	Βελτιστοποίηση	2

1 Υλοποίηση

1.1 Ανάθεση Συναρτήσεων

- Ο Ιωάννης Γρυπιώτης ασχολήθηκε με την *RobustPrune* στην οποία επιτυγχάνεται το κλάδεμα γειτώνων των εκάστοτε κόμβων βάσει της μεταβλητής α .
- Ο Ιωάννης Κούβελας ασχολήθηκε με την *GreedySearch* η οποία βρίσκει σε ένα *query*, σημείο το οποίο δεν βρίσκεται απαραίτητα στον εκάστοτε γράφο, τους k κοντινότερους γείτονες του και τους κόμβους που επισκεφτήκαμε.
- Ο Πάντιος Ματιάτος ασχολήθηκε με την *Vamana*, συνάρτηση η οποία κατασκευάζει αρχικά έναν τυχαίο κατευθυνόμενο γράφο και χρησιμοποιεί τις προηγούμενες δύο συναρτήσεις για την προσεγγιστική εύρεση του *KNNG*.

1.2 Γενικά

Στο υπόλοιπο κομμάτι της εργασίας συνεργαστήκαμε για την υλοποίηση του κώδικα και την ενσωμάτωση των συναρτήσεων για να έχουμε τα σωστά αποτελέσματα. Αυτό το καταφέραμε μιλώντας καθημερινά σε βιντεοκλήσεις ή μηνύματα βρίσκοντας λάθη και προχωρώντας με το γράψιμο των συναρτήσεων του κώδικα και των τεστ. Αξίζει να σημειωθεί ότι ο Πάντιος Ματιάτος ανέλαβε κατά κόρων την ενασχόληση με το *github* και τις ανάγκες που αυτό απαιτεί. Συγκεκριμένα διαφέρεται πως ο αριθμός των *commit* τα οποία εκτέλεσε υπερτερούν αυτά των συναδέλφων του. **Προφανώς το σύνολο κώδικα το οποίο μεταφόρτωσε δεν ήταν στην ολότητα του γραμμένο απο τον ίδιο.**

1.3 Πώς να τρέξετε το εκτελέσιμο

Για να τρέξουμε το κύριο κομμάτι κώδικα κάνουμε *make* και στη συνέχεια τρέχουμε στο *terminal* `./build > k L R a`, όπου φροντίζουμε να βάλουμε τα ορίσματα αυτά με την σωστή σειρά. Για να τρέξουμε τα *unit tests* κάνουμε *make test_all* και στη συνέχεια `./run_all_tests` για να επιβεβαιώσουμε την ορθότητα του κώδικα μας.

1.4 Επεξηγήσεις

Στον κώδικα και συγκεκριμένα στις συνάρτησεις χρησιμοποιήσαμε τα *unittest* για να ελέγξουμε την ορθότητά τους και για να αφαιρέσουμε την πιθανότητα λαθών από αυτές. Επιπλέον διαμορφώσαμε τον κώδικα με πολλά σχόλια στις συναρτήσεις για καλύτερη κατανόηση με τέτοιο τρόπο ώστε να είναι έτοιμος για την εξέλιξή του στο μέλλον από τις επόμενες εργασίες που θα μας ανατεθούν.

1.5 Σχεδιαστικές Επιλογές

Ο γράφος ο οποίος χρησιμοποιούμε στον κώδικα είναι ένα *struct Graph* μέσα στον οποίο έχουμε έναν πίνακα από *nodes*, όπου στο καθέ *node* περιέχετε ένας πίνακας από *vectors/floats* και τις ακμές που συνδέουν αυτό με τα άλλα *nodes* (οι ακμές υλοποιήθηκαν με *set* για την αποφυγή διπλότυπων). Επίσης εμπεριέχεται το *R* δηλαδή ο αριθμός των εξερχόμενων γειτόνων, και δύο ακεραίοι *number_of_nodes* και *dimension* για τις γραμμές και στήλες αντίστοιχα. Ακόμη υπάρχει και το *struct Query* που έχει τις ίδιες διαστάσεις και έναν διδιάστατο πίνακα από *floats*. Εν τέλει έχουμε το *groundtruth* που χρησιμοποιούμε για την εύρεση της αποδοτικότητας του βασικού μας αλγόριθμου και το *greedy_results* στο οποίο αποθηκεύουμε τα αποτελέσματα της συνάρτησης *GreedySearch*, δηλαδή *L, V*.

1.6 Βελτιστοποίηση

Κατά την δημιουργία του *medoid* γίνεται αποθήκευση όλων των αποστάσεων σε έναν διδιάστατο πίνακα. Αυτός χρησιμοποιείται εν συνεχεία τόσο στην *RobustPrune* όσο και στην *GreedySearch* για την αποφυγή του επανυπολογισμού των αποστάσεων. Κατά συνέπεια η απόσταση βρίσκεται σε χρονική πολυπλοκότητα της τάξης του $\Theta(1)$ αντί για $\Theta(d)$ όπου d η διάσταση του *vector*.